

Infrastructure as Code (IaC) avec PowerShell DSC

Ce document aborde PowerShell Desired State Configuration (DSC), la plateforme de gestion de configuration native de Microsoft. Il détaille le changement de paradigme entre l'approche impérative et déclarative, l'architecture de compilation (fichiers MOF), le rôle du LCM (Local Configuration Manager) et le contrôle de la dérive de configuration (*Configuration Drift*).

1. Le Changement de Paradigme : Impératif vs Déclaratif

Jusqu'à présent, vos scripts PowerShell utilisaient une approche **impérative** : vous écriviez la séquence exacte d'actions à exécuter (le *comment*).

- **Exemple impératif** : "Vérifie si le service IIS est installé. S'il ne l'est pas, installe-le. Ensuite, vérifie si le service tourne. S'il est arrêté, démarre-le."
- **Problème** : Si le script échoue au milieu, le serveur est dans un état instable. De plus, le code est long et complexe à maintenir car il doit gérer tous les cas d'erreur possibles (les blocs try/catch, les if/else).

En DevOps, l'Infrastructure as Code repose sur l'approche **déclarative**. Avec DSC (tout comme avec Ansible ou Terraform), vous décrivez uniquement l'état final attendu (le *quoi*), et le moteur interne se débrouille pour atteindre cet état.

- **Exemple déclaratif (DSC)** : "Le rôle IIS DOIT être installé et le service DOIT être en cours d'exécution."
- **Avantage** : Le code est lisible, orienté métier, et surtout **idempotent** (il peut être exécuté 1000 fois sans rien casser si l'état est déjà correct).

2. L'Architecture de PowerShell DSC

Le fonctionnement de DSC se divise en trois phases distinctes :

Phase 1 : La Rédaction (Authoring)

L'administrateur rédige un script PowerShell contenant un bloc spécial appelé Configuration. Ce bloc décrit l'état attendu du serveur en utilisant des **Ressources DSC** (ex: WindowsFeature, Service, File).

Phase 2 : La Compilation (Le fichier MOF)

Contrairement à un script classique, le bloc Configuration ne s'exécute pas directement sur le serveur. Lorsqu'on le lance en mémoire, il génère un fichier texte normé appelé **MOF** (*Managed Object Format*).

Le fichier MOF est un standard industriel indépendant de PowerShell. Il contient la "recette" exacte de la configuration à appliquer.

Phase 3 : L'Exécution via le LCM (Deployment)

Le fichier MOF est envoyé au **Local Configuration Manager (LCM)** du serveur cible. Le LCM est le moteur de DSC (intégré nativement dans WMI). C'est lui qui lit la recette, vérifie l'état actuel du serveur, et applique les modifications nécessaires en arrière-plan.

3. Structure d'une Configuration DSC

Voici la syntaxe canonique d'une configuration DSC pour provisionner un serveur Web de base :

1. Déclaration du bloc de configuration

```
Configuration ServeurWebDeBase {
```

```
    # Importation des ressources nécessaires (facultatif sous PS 5.1 mais recommandé)
```

```
    Import-DscResource -ModuleName 'PSDesiredStateConfiguration'
```

2. Définition du ou des nœuds cibles (Serveurs)

```
Node "SRV-APP01" {
```

```
    # 3. Utilisation d'une Ressource DSC (Rôle Windows)
```

```
    WindowsFeature InstallationIIS {
```

```
        Ensure = "Present"    # L'état voulu : Present (Installé) ou Absent (Désinstallé)
```

```
        Name   = "Web-Server" # Le nom technique du rôle
```

```
    }
```

4. Utilisation d'une autre Ressource DSC (Service système)

```
    Service DemarrageIIS {
```

```
        Name = "W3SVC"
```

```
        State = "Running"
```

```
        # Dépendance : On ne démarre le service QUE SI l'installation a réussi
```

```
        DependsOn = "[WindowsFeature]InstallationIIS"
```

```
    }
```

```
}
```

```
}
```

4. Cycle de Vie du Déploiement (Mode Push)

Dans un environnement simple, DSC s'utilise en mode "Push" (Pousser) depuis une console d'administration centrale.

Étape 1 : Compiler le fichier MOF

On exécute simplement la configuration en appelant son nom et en lui donnant un dossier de destination.

```
ServeurWebDeBase -OutputPath "C:\DscBuilds"
```

Résultat : PowerShell crée un dossier C:\DscBuilds contenant le fichier SRV-APP01.mof.

Étape 2 : Déployer et appliquer la configuration

On ordonne au LCM du serveur cible de lire et d'appliquer ce fichier MOF via la cmdlet Start-DscConfiguration.

```
Start-DscConfiguration -Path "C:\DscBuilds" -Wait -Verbose -Force
```

- -Path : Le dossier contenant les fichiers .mof.
- -Wait : Garde la console ouverte et affiche la progression au lieu de l'exécuter en tâche de fond.
- -Verbose : Affiche les actions exactes du LCM (ex: "Le rôle IIS n'est pas présent, installation en cours...").

Étape 3 : Vérifier l'état (Idempotence)

Si vous relancez exactement la même commande Start-DscConfiguration une seconde fois, l'exécution prendra 2 secondes. Le LCM lira le MOF, constatera que IIS est déjà là et que le service tourne, et ne fera strictement rien. C'est l'essence même de l'automatisation fiable.

5. Le Contrôle de la Dérive (Configuration Drift)

L'un des plus grands problèmes en production est la **dérive de configuration** (un administrateur se connecte un soir à 3h du matin pour dépanner, modifie une clé de registre ou désactive un service, et oublie de remettre la configuration d'origine).

DSC résout ce problème grâce au **LCM**.

Interroger l'état actuel

À tout moment, vous pouvez demander au LCM si le serveur respecte toujours son cahier des charges :

```
Test-DscConfiguration -ComputerName "SRV-APP01"
```

Retourne \$true si le serveur est conforme, \$false si quelqu'un a altéré l'état (par exemple, en

arrêtant le service IIS manuellement).

Obtenir les détails de la dérive

Si le serveur n'est plus conforme, vous pouvez identifier exactement quelle ressource pose problème :

Get-DscConfigurationStatus

L'Auto-Correction (Mode Pull / Auto-Correction)

Par défaut, le LCM est configuré en mode ApplyAndMonitor (Applique et Surveille).

Dans des architectures matures, on configure le LCM en mode ApplyAndAutoCorrect. Dans ce mode, le LCM revérifie l'état du serveur toutes les 15 minutes. S'il détecte qu'un administrateur a désinstallé IIS, **il le réinstalle automatiquement sans aucune intervention humaine** pour ramener le système à l'état désiré.

6. Synthèse des Concepts Clés pour le DevOps

1. **Le Code est la Source de Vérité (Source of Truth)** : Le script de configuration DSC est stocké sur Git. Si l'on veut modifier la production, on modifie le fichier sur Git, on valide, et la pipeline recompile le MOF. On ne modifie plus le serveur à la main.
2. **Découplage** : On sépare les *données* (le nom des serveurs, les IP) de la *logique* (le bloc Configuration).
3. **Réutilisabilité** : Les ressources DSC (File, Group, User, Registry) couvrent 99% des besoins d'infrastructure Windows, évitant de réinventer la roue avec des scripts complexes.